# Real-time Face Reenactment via SVD-based Landmark Mapping

Viktor Syrotiuk, Mykhailo Brytan, Yulian Volodymyr Zaiats

March 11, 2026

## 1 Our research area and why it is important

Our project lies in the area of computer vision and face reenactment. Face reenactment means transferring facial expressions from one face (for example, from a live video of a person) to another face (for example, a static photo or avatar). This problem is closely connected with linear algebra: images can be represented as matrices, facial landmarks as vectors in a linear space, and transformations between expressions as matrix mappings or solutions of linear systems.

This area is important for several reasons. First, it has many practical applications: virtual avatars in video calls, animation of characters in games and films, human–computer interaction, and assistive tools for people who cannot easily express emotions. Second, it is a good example of how the core topics of a linear algebra course — vector spaces, matrices, linear transformations, least squares, orthogonality, and singular value decomposition (SVD) — can be used in a real-world computer vision problem.

Our goal is to study a method that remains mathematically interpretable. Therefore, instead of relying only on heavy deep learning models, we focus on a landmark-based approach in which the main computational step is solving a least-squares problem using SVD.

## 2 Aim of the project and main tasks

The main aim of our project is:

> The aim of the project is to implement and evaluate a landmark-based face reenactment pipeline in which facial expression transfer is formulated as a least-squares problem and solved using Singular Value Decomposition (SVD), and to compare this interpretable linear-algebra-based approach with a neural-network-based baseline.

It is important to distinguish the two notions:

- **Least squares** is the mathematical optimization problem: we want to minimize the total squared mismatch between predicted and observed landmark positions.
- **SVD** is the numerical linear algebra tool that we use to solve this least-squares problem in a stable way, especially when the system is overdetermined or nearly rank-deficient.

To achieve this aim, we divide the project into the following tasks:

1. Represent each face by a set of detected 2D landmarks and store them as vectors or matrices.
2. Build a linear model that maps landmarks of the source face to landmarks of the target expression.
3. Estimate the unknown transformation by solving a least-squares problem via SVD.
4. Compare two implementation strategies:
   - a purely linear-algebra pipeline without a neural network,
   - a pipeline that uses a neural model as a baseline or extension.
5. Evaluate the resulting reenactment using both geometric and visual quality metrics.

## Algorithmic outline

### Algorithm 1: Landmark-based reenactment without a neural network

1. Input a static source image $I_s$ and a live target video stream $\{I_t\}$.
2. Detect facial landmarks on the source image and form the source landmark vector

$$x_s \in \mathbb{R}^{2k},$$

   where $k$ is the number of landmarks.
3. For each calibration frame $t_i$:
   (a) detect target landmarks $y_{t_i} \in \mathbb{R}^{2k}$;
   (b) construct equations relating $x_s$ to $y_{t_i}$;
   (c) stack all equations into a system

$$Ma \approx b,$$

   where $a$ contains the unknown parameters of the transformation.

4. Compute the least-squares solution

$$\hat{a} = \arg\min_{a} \|Ma - b\|_2^2$$

using SVD of $M$.

5. For each new frame:

   (a) detect current target landmarks $y_t$;

   (b) predict mapped landmarks $\hat{y}_t$ using the estimated transformation;

   (c) warp the source image so that its landmarks move from $x_s$ to $\hat{y}_t$;

   (d) display the reenacted frame.

**Algorithm 2: Reenactment with a neural network baseline**

1. Input a source image and a target video.
2. Detect landmarks or dense facial motion features.
3. Feed the source image together with motion/expression descriptors into a neural reenactment model.
4. Generate the reenacted frame with the neural network.
5. Compare the output against the landmark-based linear model using the same evaluation metrics.

Thus, the project is not only about implementing one method, but also about comparing an interpretable linear method with a more expressive neural approach.

# 3 Approaches and available solutions

Face reenactment is a known problem in computer vision, and there are two broad classes of approaches:

1. **classical landmark-based or geometry-based methods,**
2. **neural methods based on deep generative models.**

## 3.1 Landmark-based linear modeling

A simple and interpretable approach is to represent a face by a vector of landmark coordinates. If there are $k$ landmarks, each with coordinates $(x_i, y_i)$, then one frame can be written as a vector in $\mathbb{R}^{2k}$:

$$x = (x_1, y_1, x_2, y_2, \ldots, x_k, y_k)^\top.$$

Then the expression transfer problem can be approximated by a linear relation

$$y \approx Ax,$$

where $x$ is the source landmark vector, $y$ is the target landmark vector, and $A$ is a matrix describing the transformation between the two expressions.

This model is appealing because:

- each coordinate of $y$ gives a linear equation in the unknown entries of $A$;
- the matrix $A$ is a standard linear transformation between vector spaces;
- the rank of $A$ indicates how many independent modes of facial motion are effectively captured.

In practice, the equations are not exact, because landmark detection is noisy and facial motion is only approximately linear. Therefore, instead of solving $Ax = y$ exactly, we solve an overdetermined least-squares system.

## 3.2 Least squares and SVD

Suppose that from many frames we build a system

$$Ma \approx b,$$

where $a$ is the vector of unknown transformation parameters. Usually, the number of equations is larger than the number of unknowns, so the system has no exact solution. Then we solve

$$\hat{a} = \arg\min_a \|Ma - b\|_2^2.$$

This is the **least-squares problem**. To compute $\hat{a}$ robustly, we use the singular value decomposition

$$M = U\Sigma V^\top.$$

Then the least-squares solution can be written using the pseudoinverse:

$$\hat{a} = M^+ b = V\Sigma^+ U^\top b.$$

So, least squares answers the question *what* we optimize, while SVD gives a numerically stable answer to *how* we solve it.

## 3.3 Landmark detectors

A key part of the whole pipeline is the landmark detector. If landmark positions are inaccurate, then even a mathematically correct SVD solution will be based on noisy data. Therefore, detector quality directly affects reenactment quality.

Examples of commonly used landmark detectors include:

- **OpenFace** — a classical open-source facial behavior analysis toolkit that includes facial landmark tracking;
- **MediaPipe Face Mesh / Attention Mesh** — modern real-time detectors that provide dense facial landmarks and are widely used in practical systems.

These detectors are useful in our project because they convert the visual problem into a linear algebra problem: once the landmarks are detected, the rest of the pipeline can be described in terms of vectors, matrices, least squares, and transformations.

## 3.4 Methods without a neural network

There exist geometry-driven reenactment systems that do not rely on full neural generation. A well-known example is **Face2Face**, where facial motion is transferred by estimating and tracking facial geometry in real time. Such methods are closer in spirit to our project because they explicitly model facial structure and motion rather than learning the whole mapping end-to-end.

Our own non-neural approach belongs to this family: we detect landmarks, estimate a transformation from the source face to the target expression, and then warp the source image accordingly.

## 3.5 Methods with a neural network

Modern face reenactment is often based on deep learning. In neural methods, a model learns a nonlinear mapping from the source appearance and the driving expression to the target reenacted frame. Such approaches usually produce more realistic textures and handle complicated deformations better than a single linear map.

Typical examples include:

- **First Order Motion Model (FOMM)** for image animation,
- **One-Shot Free-View Neural Talking-Head Synthesis**,
- more recent diffusion-based or GAN-based reenactment systems.

These models are powerful, but they are harder to interpret mathematically. For our report, they serve as an important comparison point: they show what can be achieved with nonlinear learned mappings, while our method highlights the role of linear algebra and interpretability.

## 3.6 Literature where this is used

To motivate the project, we note that related ideas are used in several lines of research:

- facial landmark detection and tracking for facial behavior analysis;

- geometry-based facial reenactment and expression transfer;
- neural talking-head synthesis and image animation.

Representative references include:

1. T. Baltrušaitis, P. Robinson, and L.-P. Morency, *OpenFace: An open source facial behavior analysis toolkit*, WACV, 2016.
2. T. Kartynnik et al., *Real-time facial surface geometry from monocular video on mobile GPUs*, arXiv, 2019. (Attention Mesh / MediaPipe line)
3. J. Thies et al., *Face2Face: Real-time face capture and reenactment of RGB videos*, CVPR, 2016.
4. A. Siarohin et al., *First Order Motion Model for Image Animation*, NeurIPS, 2019.
5. E. Zakharov et al., *One-Shot Free-View Neural Talking-Head Synthesis for Video Conferencing*, CVPR, 2019.

These papers show that landmark detection, facial geometry transfer, and reenactment are active and important research directions.

# 4 Pseudocode of the implementation

Below we summarize the implementation more formally.

## 4.1 Pipeline A: without neural network

**Input:** static source image $I_s$, live video stream $V(t)$

**Output:** reenacted video frames $\hat{I}(t)$

1. Detect landmarks on the static source image:
   (a) use a landmark detector to obtain $k$ facial points on $I_s$;
   (b) store them as a vector $x_s \in \mathbb{R}^{2k}$.
2. Calibration stage:
   (a) for each selected calibration frame $t_i$, detect target landmarks $y_{t_i} \in \mathbb{R}^{2k}$;
   (b) construct the linear system $Ma \approx b$ from the source-target correspondences;
   (c) compute the least-squares estimate $\hat{a}$ using SVD.
3. Real-time stage:
   (a) for each new frame $t$, detect target landmarks $y_t$;
   (b) predict reenacted landmarks $\hat{y}_t$ from the fitted mapping;
   (c) warp the pixels of the source image so that $x_s \mapsto \hat{y}_t$;
   (d) output the reenacted frame $\hat{I}(t)$.

### 4.2 Pipeline B: with neural network

1. Detect landmarks or motion descriptors from the target frame.
2. Encode source appearance and target motion.
3. Use a neural generator to synthesize the reenacted face.
4. Compare the neural output with the output of Pipeline A.

### 4.3 Why we compare both versions

The non-neural version is simpler and more interpretable. Every step can be explained with standard linear algebra tools. The neural version is more flexible and may produce more realistic outputs, but it introduces nonlinear black-box components. Comparing them helps answer an important question of the project: how far can a clean linear algebra method go before one needs a neural model?

## 5 Testing strategy, metrics, and data

To test the implementation, we will first collect a small custom dataset consisting of short videos with several facial expressions (neutral, smiling, surprised, etc.) together with one neutral source image. If necessary, we will additionally use small public facial-expression or talking-face datasets.

### 5.1 Metrics for landmark accuracy

The report must include quantitative metrics, not only visual examples. Since our method is based on landmarks, the first group of metrics should measure how well the predicted landmarks match the target landmarks.

- **Mean Euclidean Landmark Error**

$$\frac{1}{k}\sum_{i=1}^{k}\|\hat{p}_i - p_i\|_2,$$

  where $p_i$ is the true target landmark and $\hat{p}_i$ is the predicted one.

- **Normalized Mean Error (NME)** — the average landmark error normalized by a face size measure (for example, inter-ocular distance or bounding-box size).
- **CED curve (Cumulative Error Distribution)** — shows the fraction of samples whose landmark error is below a threshold.
- **AUC of the CED curve** — summarizes the error distribution into a single number.

- **Failure rate** — the proportion of frames for which the landmark error exceeds a fixed threshold.

These metrics are standard in landmark detection and alignment papers, so they are appropriate for our project as well.

### 5.2 Metrics for reenacted image quality

Since the final output is an image or video frame, it is also useful to evaluate image quality and realism. For this purpose, we can use:

- **PSNR** (Peak Signal-to-Noise Ratio),
- **SSIM** (Structural Similarity Index),
- **LPIPS** (Learned Perceptual Image Patch Similarity),
- **FID** (Fréchet Inception Distance), when comparing distributions of generated and reference images,
- **Identity similarity**, if we want to check whether the identity of the source face is preserved.

For motion transfer papers, one may also consider landmark-distance-based motion consistency measures, because they directly reflect how well the expression has been transferred.

### 5.3 Visual evaluation

In addition to metrics, we will visually inspect:

- whether the reenacted face follows the target expression,
- whether the motion looks smooth from frame to frame,
- whether distortions appear around the mouth, eyes, or jaw.

Thus, the evaluation should combine **geometric accuracy**, **image quality**, and **visual plausibility**.

## 6 Plan of future research

We propose the following schedule:

- **Week 1:** Implement data loading and landmark extraction pipeline. Prepare a small set of test videos and source images.

- **Week 2:** Formulate the linear least-squares system for the transformation matrix and implement the SVD-based solver.
- **Week 3:** Integrate the linear mapping into a simple real-time or near real-time reenactment loop.
- **Week 4:** Add a neural baseline or compare with an available neural reenactment model.
- **Week 5:** Evaluate the system with quantitative metrics and finalize the report and presentation.

# 7 Possible challenges

Several challenges may affect the project:

- **Landmark detection errors.** If the detector is inaccurate, then the estimated linear system is noisy, and the SVD solution may fit wrong correspondences.
- **Limits of a linear model.** Real facial deformations are not perfectly linear. A single matrix may not fully capture complex expressions, self-occlusion, or large head rotations.
- **Warping artifacts.** Even if the mapped landmarks are accurate, the image warping step may create visible distortions.
- **Real-time performance.** The linear algebra part is relatively efficient, but total runtime also depends on the speed of video capture, landmark detection, and image warping.
- **Comparison fairness.** Neural methods and non-neural methods may have different computational budgets and data requirements, so comparison must be interpreted carefully.

# 8 Conclusion

Our project provides a clear connection between linear algebra and a modern computer vision application. The central mathematical problem is a least-squares fit of landmark transformations, while SVD is the main computational tool used to solve this problem robustly. At the same time, the project allows us to compare a transparent linear method with more powerful but less interpretable neural approaches.

This makes the project suitable both as an application of course material and as a small research study in face reenactment.

# References


1. T. Baltrušaitis, P. Robinson, L.-P. Morency, *OpenFace: An open source facial behavior analysis toolkit*, IEEE Winter Conference on Applications of Computer Vision (WACV), 2016.

2. T. Kartynnik, Y. Ablavatski, I. Grishchenko, M. Grundmann, *Real-time Facial Surface Geometry from Monocular Video on Mobile GPUs*, arXiv preprint arXiv:1907.06724, 2019.

3. J. Thies, M. Zollhöfer, M. Stamminger, C. Theobalt, M. Nießner, *Face2Face: Real-time Face Capture and Reenactment of RGB Videos*, IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.

4. A. Siarohin, S. Lathuilière, S. Tulyakov, E. Ricci, N. Sebe, *First Order Motion Model for Image Animation*, Advances in Neural Information Processing Systems (NeurIPS), 2019.

5. E. Zakharov, A. Ivakhnenko, A. Shysheya, V. Lempitsky, *One-Shot Free-View Neural Talking-Head Synthesis for Video Conferencing*, IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2019.